

Mlvan Rojas's Org
Scan Report



Application: juventudes2025
Environment: Pre-Production
Completed On: Wed, 30 Jul 2025 20:07:38 GMT

Application: juventudes2025
Environment: Pre-Production

Scan Date: Wed, 30 Jul 2025 20:07:38 GMT | Paths Scanned: 9 | Scan Duration: 131s

High	Medium	Low	New	Assigned	Risk Accepted	False Positive
0	7	4	11	0	0	0

Criticality	CWE	Finding	New	Assigned	Risk Accepted	False Positive
Medium	CWE-693	CSP: script-src unsafe-inline	1	0	0	0
Medium	CWE-693	CSP: Wildcard Directive	3	0	0	0
Medium	CWE-1021	Multiple X-Frame-Options Header Entries	3	0	0	0
Low	CWE-693	Permissions Policy Header Not Set	4	0	0	0

CSP: script-src unsafe-inline

Criticality: Medium CWE ID: 693

Description:

Remediation

To fix the vulnerability "CSP: script-src unsafe-inline", you need to update your Content Security Policy (CSP) to disallow the use of inline scripts. This can be done by removing the "unsafe-inline" keyword from the "script-src" directive.

Here is an example of a CSP header that disallows inline scripts:

```
Content-Security-Policy: script-src 'self' 'unsafe-eval';
```

In this example, the "unsafe-inline" keyword has been replaced with "unsafe-eval" to allow the use of eval() function, but inline scripts are still disallowed.

About

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page.

The "CSP: script-src unsafe-inline" vulnerability occurs when the CSP allows the use of inline scripts using the "unsafe-inline" keyword in the "script-src" directive. Inline scripts are considered unsafe because they can be easily manipulated by attackers to execute malicious code on the website.

Risks

The "CSP: script-src unsafe-inline" vulnerability poses several risks to your website:

- 1. Cross Site Scripting (XSS):** Allowing inline scripts increases the risk of XSS attacks. Attackers can inject malicious scripts into the website, which can lead to the theft of sensitive user information or the manipulation of website content.
- 2. Data Injection Attacks:** Inline scripts can be used to inject malicious code into the website's data, leading to data corruption or unauthorized access to sensitive information.
- 3. Malware Distribution:** Attackers can use inline scripts to distribute malware to website visitors. This can result in the compromise of user devices and the spread of malware to other systems.

It is important to fix this vulnerability by updating your CSP to disallow inline scripts and ensure the security of your website and its users.

Path	Method	Status
/sitemap.xml	GET	NEW

CSP: Wildcard Directive

Criticality: Medium CWE ID: 693

Description:

Remediation

To mitigate the vulnerability of the "CSP: Wildcard Directive," website owners should follow these steps:

- 1. Specify allowed sources:** Instead of using a wildcard directive (*) in the Content Security Policy (CSP) header, explicitly define the approved sources for each content type. This ensures that only trusted sources are allowed to load on the page.

Example:

```
Content-Security-Policy: script-src 'self' https://trusted-domain.com;
```

- 2. Regularly review and update CSP:** Periodically review the CSP configuration to ensure it aligns with the website's requirements. Update the policy as needed to accommodate any changes in trusted sources or content types.
- 3. Implement a strict CSP:** Consider implementing a strict CSP that only allows trusted sources and restricts the use of inline scripts or unsafe practices. This can provide an additional layer of protection against various types of attacks.

Example:

```
Content-Security-Policy: default-src 'none'; script-src 'self' https://trusted-domain.com; style-src 'self' https://trusted-domain.com; img-src 'self' https://trusted-domain.com;
```

About

Content Security Policy (CSP) is an added layer of security that helps detect and mitigate certain types of attacks, such as Cross-Site Scripting (XSS) and data injection attacks. It provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on a page. The covered types include JavaScript, CSS, HTML frames, fonts, images, and embeddable objects like Java applets, ActiveX, audio, and video files.

Risks

The vulnerability of the "CSP: Wildcard Directive" arises when a website owner uses a wildcard (*) directive in the Content Security Policy (CSP) header. This allows any source to load content on the page, which can lead to various risks:

- 1. Increased attack surface:** Allowing any source to load content increases the attack surface, making it easier for malicious actors to inject malicious scripts or content into the page.
- 2. Cross-Site Scripting (XSS) attacks:** Without proper source restrictions, XSS attacks can be executed by injecting malicious scripts from untrusted sources.
- 3. Data theft and site defacement:** Malicious actors can exploit the wildcard directive to steal sensitive data or deface the website, impacting its reputation and user trust.
- 4. Distribution of malware:** Allowing content from untrusted sources increases the risk of distributing malware to website visitors, potentially compromising their devices and data.

Path	Method	Status
/sitemap.xml	GET	NEW

Multiple X-Frame-Options Header Entries

Criticality: Medium CWE ID: 1021

Description:

Remediation

To remediate the vulnerability of "Multiple X-Frame-Options Header Entries," you should ensure that only one X-Frame-Options header is included in the response. This can be achieved by removing any additional X-Frame-Options headers that may be present.

About

X-Frame-Options (XFO) headers are used to protect against clickjacking attacks by specifying whether a web page can be displayed within an iframe. When multiple XFO headers are present in a response, it can lead to unpredictable behavior in different user-agents (browsers). This vulnerability occurs when a web application includes multiple X-Frame-Options headers in its HTTP response.

Risks

Having multiple X-Frame-Options header entries can introduce the following risks:

- 1. Inconsistent behavior:** Different user-agents may interpret and handle multiple XFO headers differently, leading to inconsistent behavior across browsers. This can result in unexpected display or functionality issues for users.
- 2. Weakened clickjacking protection:** Multiple XFO headers can potentially weaken the clickjacking protection provided by X-Frame-Options. If a user-agent chooses to ignore or override one of the headers, it may allow an attacker to successfully perform clickjacking attacks on the web application.
- 3. Confusion and misconfiguration:** Including multiple XFO headers can indicate a misconfiguration or confusion in the web application's security settings. This can make it difficult to understand and manage the intended clickjacking protection for the application.

To mitigate these risks, it is important to ensure that only one X-Frame-Options header is included in the response. Here is an example of how to set the X-Frame-Options header in different web server configurations:

•**Apache:** Add the following line to your Apache configuration file or .htaccess file:

```
Header always set X-Frame-Options SAMEORIGIN
```

•**Nginx:** Add the following line to your Nginx configuration file:

```
add_header X-Frame-Options SAMEORIGIN;
```

•**IIS:** Open the IIS Manager, select the website or application, and go to the "HTTP Response Headers" section. Add a new header with the name "X-Frame-Options" and the value "SAMEORIGIN".

By ensuring that only one X-Frame-Options header is present in the response, you can mitigate the vulnerability of "Multiple X-Frame-Options Header Entries" and improve the consistency and effectiveness of clickjacking protection for your web application.

Path	Method	Status
[ROOT]	GET	NEW
/robots.txt	GET	NEW
/sitemap.xml	GET	NEW

Permissions Policy Header Not Set

Criticality: Low CWE ID: 693

Description:

Remediation

To fix the "Permissions Policy Header Not Set" vulnerability, you need to add the Permissions-Policy header to your web server configuration. This header allows you to specify which browser features can be used by your web resources. By setting appropriate permissions, you can restrict unauthorized access or usage of sensitive features such as camera, microphone, location, and full screen.

To add the Permissions-Policy header, follow these steps:

1. Apache: Add the following line to your .htaccess file or Apache configuration file:

```
Header set Permissions-Policy "camera=(), microphone=(),  
geolocation=(), fullscreen=()"
```

2. Nginx: Add the following line to your Nginx configuration file within the http block:

```
add_header Permissions-Policy "camera=(), microphone=(),  
geolocation=(), fullscreen=()";
```

3. IIS: Open the IIS Manager, select your website, and go to the "HTTP Response Headers" feature. Add a new header with the name "Permissions-Policy" and the value "camera=(), microphone=(), geolocation=(), fullscreen=()".

Make sure to adjust the permissions according to your specific requirements. For example, if your website requires camera access, you can remove the camera=() part from the header value.

About

The Permissions Policy Header is an added layer of security that helps restrict unauthorized access or usage of browser/client features by web resources. It ensures user privacy by limiting or specifying which features of the browser can be used by web resources. By setting the Permissions-Policy header, website owners can control access to features such as camera, microphone, location, and full screen.

Risks

If the Permissions Policy Header is not set, it can lead to potential security risks. Without proper restrictions, malicious actors may exploit browser features to gain unauthorized access to sensitive user information or perform unauthorized actions. For example, an attacker could abuse

camera or microphone access to capture audio/video without user consent, or misuse geolocation data for tracking purposes. By not setting the Permissions Policy Header, you leave your website and users vulnerable to these risks.

Path	Method	Status
/sitemap.xml	GET	NEW